EXPERIMENT 2 – DATA DISCOVERY AND PREPARATION

Learning Objective

The objective of this experiment is to perform data discovery and exploratory data analysis on a real-world dataset.

Problem Statement

A telecommunications company has observed an increase in customer churn, leading to

revenue loss. The management wants to understand customer behavior and identify patterns before developing a machine learning model to predict churn.

As a Data Analyst, your responsibility is to discover meaningful insights from the dataset,

identify data quality issues, perform exploratory data analysis, detect outliers, and prepare the dataset for predictive analytics.

Tasks to Perform

1. Reuse Ex:1 Steps 1- 6, 10.
2. Analyze the distribution of: Customer Churn, Gender, Contract Type, Payment Method.
3. Visualize the distribution of: MonthlyCharges, TotalCharges Tenure using Histograms
1. and Boxplots.
4. Detect outliers in numerical features using the Interquartile Range (IQR) method.
5. Create suitable visualizations (Count Plot, Box Plot, Histogram, or Scatter Plot) to study
2. the relationship between: Contract Type and Churn, Internet Service and Churn,
3. Monthly Charges and Churn, Tenure and Churn.
6. Do the below analytical questions:
    a. Which contract type has the highest customer churn?
    b. Which payment method is most frequently used by customers who churn?
    c. Do customers with shorter tenure tend to churn more often?
    d. Which customer group has the highest monthly charges?
    e. Which Internet Service category experiences the highest churn?
    f. Is there any relationship between monthly charges and churn?
    g. Which features appear to be the strongest indicators of customer churn?
7. Split the cleaned dataset into Training (80%) and Testing (20%) datasets.
8. Save the final processed dataset as a new CSV file named CustomerChurn_Preprocessed.csv.

AIM To perform exploratory data analysis on the customer churn dataset, identify data quality issues and outliers, discover churn patterns, and prepare the dataset for predictive analytics.

ALGORITHM

1. Import libraries and load the dataset from a URL.
2. Inspect and clean the dataset, handle missing values, encode categorical features, and scale numerical features.
3. Analyze categorical and numerical distributions using plots.
4. Detect outliers using the IQR method.
5. Visualize relationships between customer features and churn.
6. Analyze churn patterns and identify important indicators.
7. Split the dataset into 80% training and 20% testing data.
8. Save the processed dataset as a CSV file.

PROGRAM AND OUTPUT

```
# Question 1: Reuse Experiment 1 Steps 1–6 and 10: load, inspect,
clean, encode, scale, and visualize the dataset.
# Luei johncena (241801140)
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import requests
from io import BytesIO
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split


import pandas as pd
import requests
from io import BytesIO

urls = [

"https://raw.githubusercontent.com/aiplanethub/Datasets/master/WA_Fn-
UseC_-Telco-Customer-Churn.csv",

"https://raw.githubusercontent.com/IBM/Customer-Churn-Analysis/master/
data/Telco-Customer-Churn.csv"
]

df = None

for url in urls:
    try:
        response = requests.get(url, timeout=15)
        response.raise_for_status()
        df = pd.read_csv(BytesIO(response.content))
        if "Churn" in df.columns:
            print("Dataset loaded successfully!")
            break
        df = None
    except Exception as e:
        print("URL failed:", e)

if df is None:
    raise RuntimeError("Dataset download failed. Check your internet
connection.")

print("Dataset shape:", df.shape)
display(df.head())

df = pd.read_csv(url)
display(df.head(10))
display(df.tail(10))
```

```
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
display(df.describe(include="all"))
print("Missing values:\n", df.isnull().sum())
print("Duplicates:", df.duplicated().sum())

df = df.drop_duplicates().copy()
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"],
errors="coerce")
for col in df.columns:
    if df[col].isnull().any():
        if pd.api.types.is_numeric_dtype(df[col]):
            df[col] = df[col].fillna(df[col].median())
        else:
            df[col] = df[col].fillna(df[col].mode()[0])

df_encoded = pd.get_dummies(df, drop_first=True, dtype=int)
numeric_cols = ["tenure", "MonthlyCharges", "TotalCharges"]
df_encoded[numeric_cols] =
StandardScaler().fit_transform(df_encoded[numeric_cols])
print("Encoded dataset shape:", df_encoded.shape)
display(df_encoded.head())
```

```
Dataset loaded successfully!
Dataset shape: (7043, 21)
```

{"type":"dataframe"}

{"type":"dataframe"}

{"type":"dataframe"}

```
Shape: (7043, 21)
Columns: ['customerID', 'gender', 'SeniorCitizen', 'Partner',
'Dependents', 'tenure', 'PhoneService', 'MultipleLines',
'InternetService', 'OnlineSecurity', 'OnlineBackup',
'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies',
'Contract', 'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges',
'TotalCharges', 'Churn']
```

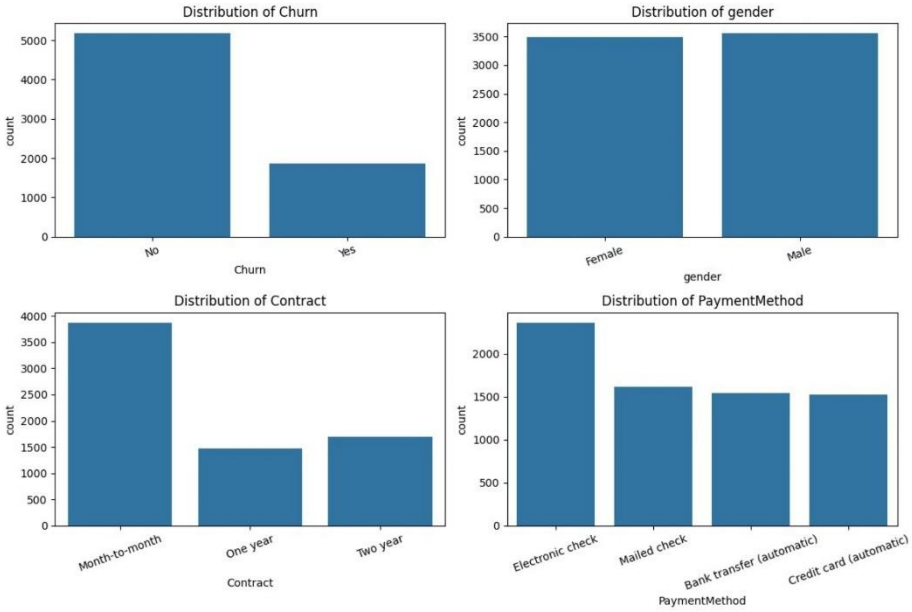
{"type":"dataframe"}

```
Missing values:
 customerID          0
gender               0
SeniorCitizen        0
Partner              0
Dependents           0
tenure               0
PhoneService         0
MultipleLines        0
```

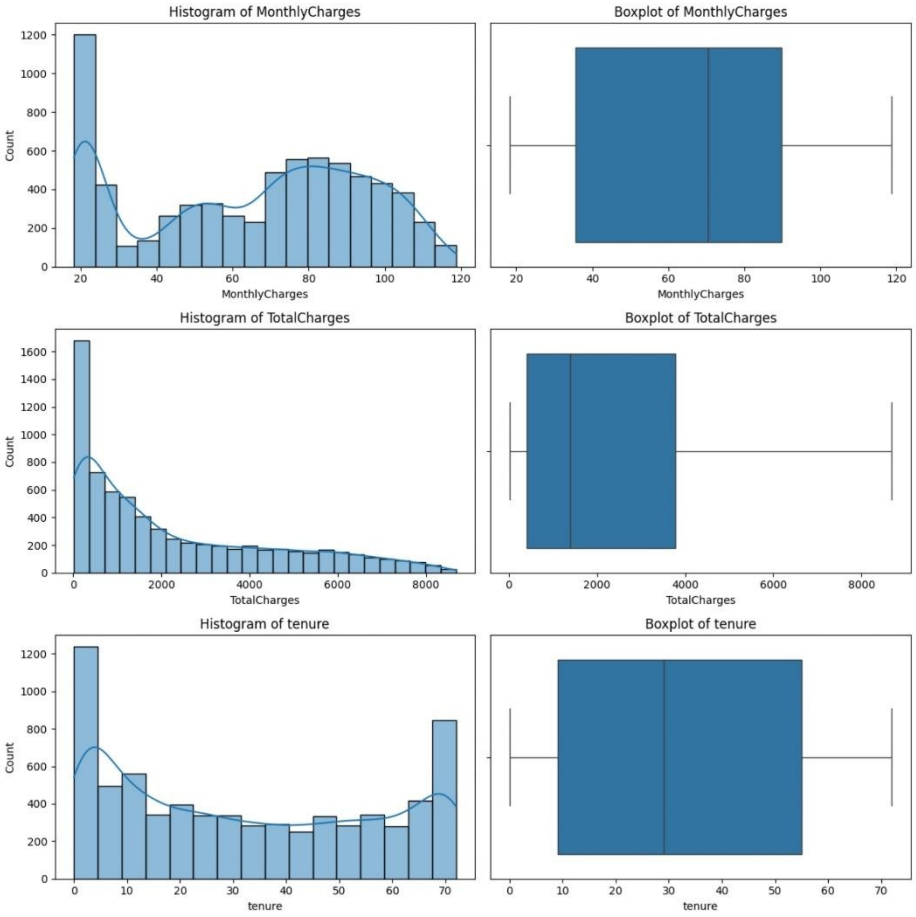
```
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
Duplicates: 0
Encoded dataset shape: (7043, 7073)
```

{"type":"dataframe"}

```
# Question 2: Analyze the distribution of Customer Churn, Gender,
Contract Type, and Payment Method.
# Luei johncena (241801140)
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for col, ax in zip(["Churn", "gender", "Contract", "PaymentMethod"],
axes.flatten()):
    sns.countplot(data=df, x=col, ax=ax)
    ax.set_title(f"Distribution of {col}")
    ax.tick_params(axis="x", rotation=20)
plt.tight_layout()
plt.show()
```



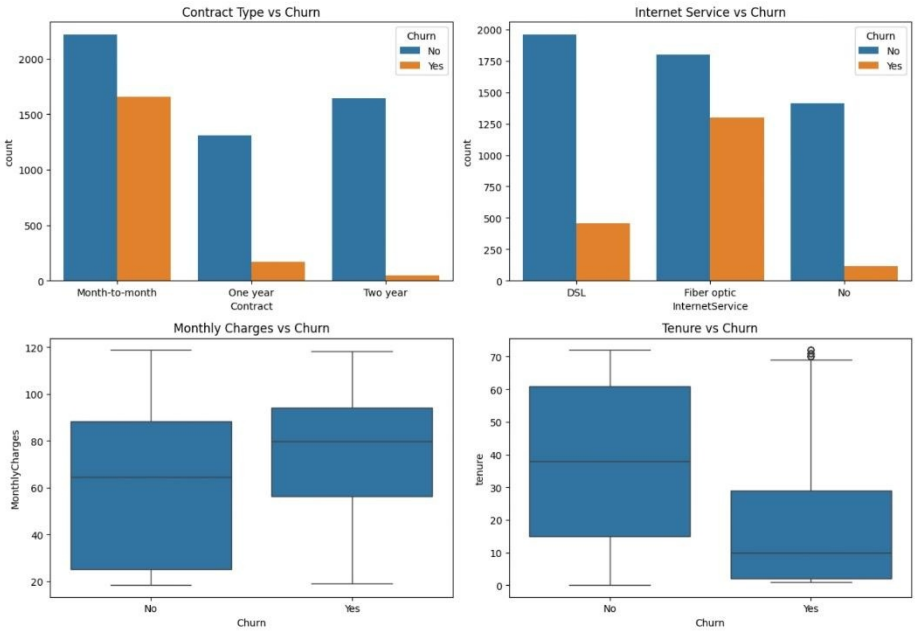

```
# Question 3: Visualize MonthlyCharges, TotalCharges, and Tenure using
histograms and boxplots.
# Luei johncena (241801140)
num_cols = ["MonthlyCharges", "TotalCharges", "tenure"]
fig, axes = plt.subplots(3, 2, figsize=(12, 12))
for i, col in enumerate(num_cols):
    sns.histplot(data=df, x=col, kde=True, ax=axes[i, 0])
    axes[i, 0].set_title(f"Histogram of {col}")
    sns.boxplot(data=df, x=col, ax=axes[i, 1])
    axes[i, 1].set_title(f"Boxplot of {col}")
plt.tight_layout()
plt.show()
```




```
# Question 4: Detect outliers in numerical features using the
Interquartile Range (IQR) method.
# Luei johncena (241801140)
for col in ["tenure", "MonthlyCharges", "TotalCharges"]:
    q1, q3 = df[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    count = ((df[col] < lower) | (df[col] > upper)).sum()
    print(f"{col}: {count} outliers (lower={lower:.2f},
upper={upper:.2f})")
```

```
tenure: 0 outliers (lower=-60.00, upper=124.00)
MonthlyCharges: 0 outliers (lower=-46.02, upper=171.38)
TotalCharges: 0 outliers (lower=-4674.34, upper=8863.16)
```

```
# Question 5: Visualize Contract Type vs Churn, Internet Service vs
Churn, Monthly Charges vs Churn, and Tenure vs Churn.
# Luei johncena (241801140)
fig, axes = plt.subplots(2, 2, figsize=(13, 9))
sns.countplot(data=df, x="Contract", hue="Churn", ax=axes[0, 0])
axes[0, 0].set_title("Contract Type vs Churn")
sns.countplot(data=df, x="InternetService", hue="Churn", ax=axes[0,
1])
axes[0, 1].set_title("Internet Service vs Churn")
sns.boxplot(data=df, x="Churn", y="MonthlyCharges", ax=axes[1, 0])
axes[1, 0].set_title("Monthly Charges vs Churn")
sns.boxplot(data=df, x="Churn", y="tenure", ax=axes[1, 1])
axes[1, 1].set_title("Tenure vs Churn")
plt.tight_layout()
plt.show()
```




```
# Question 6: Answer the analytical questions about churn patterns and
possible indicators.
# Luei johncena (241801140)
```

```
churned = df[df["Churn"].astype(str).str.lower() == "yes"]
print("Contract type with highest churn count:",
churned["Contract"].value_counts().idxmax())
print("Most used payment method among churned customers:",
churned["PaymentMethod"].value_counts().idxmax())
print("\nAverage tenure by churn status:\n", df.groupby("Churn")
["tenure"].mean())
print("\nAverage monthly charges by contract:\n",
df.groupby("Contract")
["MonthlyCharges"].mean().sort_values(ascending=False))
print("\nInternet service with highest churn count:",
churned["InternetService"].value_counts().idxmax())
print("\nAverage monthly charges by churn status:\n",
df.groupby("Churn")["MonthlyCharges"].mean())
```

```
Contract type with highest churn count: Month-to-month
Most used payment method among churned customers: Electronic check

Average tenure by churn status:
 Churn
No     37.569965
Yes    17.979133
Name: tenure, dtype: float64

Average monthly charges by contract:
 Contract
Month-to-month    66.398490
One year          65.048608
Two year          60.770413
Name: MonthlyCharges, dtype: float64

Internet service with highest churn count: Fiber optic

Average monthly charges by churn status:
 Churn
No     61.265124
Yes    74.441332
Name: MonthlyCharges, dtype: float64
```

```
# Question 7: Split the cleaned dataset into Training (80%) and
Testing (20%) datasets.
# Luei johncena (241801140)
X = df_encoded.drop(columns=["Churn_Yes"], errors="ignore")
y = df["Churn"].map({"No": 0, "Yes": 1})
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print("X_train:", X_train.shape, "X_test:", X_test.shape)
print("y_train:", y_train.shape, "y_test:", y_test.shape)
```

```
X_train: (5634, 7072) X_test: (1409, 7072)
y_train: (5634,) y_test: (1409,)

# Question 8: Save the final processed dataset as
CustomerChurn_Preprocessed.csv.
# Luei johncena (241801140)
df_encoded.to_csv("CustomerChurn_Preprocessed.csv", index=False)
print("Saved CustomerChurn_Preprocessed.csv")

Saved CustomerChurn_Preprocessed.csv
```

RESULT: -

Thus, the classification models were successfully trained and evaluated on the churn dataset, with the Random Forest model achieving the best overall performance.